

FPS PVP Demo

<https://github.com/ChaosWorldPlz/FPS>

基于 Unreal Engine 5.4 的团队对抗 FPS Demo。

架构原则：**C++ 铺底层框架 + 极少蓝图（仅资产配置） + UnLua 热更全部 UI 和玩法逻辑。**

技术栈

层	技术	职责
引擎	UE 5.4	渲染、物理、网络传输
C++	GameMode / Character / GAS / Weapon	框架、数据结构、网络同步、权威逻辑
Lua	UnLua 插件 + Lua 5.4	UI 展示、菜单流程、玩法脚本、热更新
蓝图	仅 BP_FirstPersonCharacter 等资产	Mesh/动画/特效配置

目录结构

```
1 FPS/
2 |─ Content/
3 |   |─ FirstPerson/           # 角色、地图资产
4 |   |─ Script/               # Lua 脚本（UnLua 热更）
5 |   |   |─ Core/             # 工具层（TextManager 等）
6 |   |   |─ UI/HUD/           # HUD 绑定
7 |   |   |─ UI/Inventory/     # 背包 UI
8 |   |   |─ UI/Menu/         # 菜单 UI（主菜单/暂停/设置/地图选择/装备/记分板）
9 |   |   └─ _FPS/System/UI/Menu/ # widget Blueprint 资产
10 |
11 |─ Plugins/
12 |   └─ UnLua/                # UnLua 插件（Lua 5.4）
13 |
14 |─ Source/FPS/
15 |   |─ FPS.Build.cs          # 模块依赖
16 |   |─ FPSCharacter          # 角色：1P/3P 网格、GAS、阵营接口、网络复制
17 |   |─ FPSGameMode           # 游戏模式：阵营分配、重生、计分、胜利条件
18 |   |─ FPSPlayerController   # 玩家控制器：菜单、会话管理（Host/Join/Leave）
19 |   |─ FPSProjectile         # 投射物
20 |   |
21 |   |─ Team/                 # === 阵营系统 ===
22 |   |   |─ FPSTeamTypes      # EFPSTeam 枚举 + 工具函数库
23 |   |   |─ FPSPlayerState    # 玩家状态（阵营/K-D-A/分数/伤害统计）
24 |   |   └─ FPSGameState      # 游戏状态（比赛阶段/队伍分数/倒计时）
25 |   |
26 |   |─ GAS/                  # === Gameplay Ability System ===
27 |   |   |─ FPSAbilitySystemComponent # ASC 封装（Mixed 复制模式）
28 |   |   |─ FPSAttributeSetBase      # 属性集基类
29 |   |   |─ FPSCombatAttributeSet    # 战斗属性（血量/护甲/体力 + 伤害统计）
30 |   |   |─ FPSGameplayAbility       # 能力基类
31 |   |   └─ FPSGameplayTags          # 原生 Tag 注册（状态/能力/效果/阵营）
```

```

32 |   └─ Abilities/                # 具体能力
33 |       └─ GA_WeaponFire        #  射击（含自动射击）
34 |       └─ GA_WeaponReload     #  换弹
35 |       └─ GA_WeaponMelee      #  近战
36 |       └─ GA_UseItem           #  使用物品
37 |
38 | └─ Weapon/                    # === 武器系统 ===
39 |     └─ FPSWeaponTypes        #  枚举（类型/开火模式/状态）+ 弹药结构
40 |     └─ FPSWeaponDataAsset    #  数据驱动武器配置（DataAsset）
41 |     └─ FPSWeaponBase        #  武器基类（射击/换弹/RPC/阵营过滤）
42 |
43 | └─ Inventory/                # === 网格背包系统 ===
44 |     └─ InventoryTypes       #  背包数据结构
45 |     └─ InventoryGridComponent #  网格背包逻辑组件
46 |     └─ ItemDataManager      #  物品数据管理
47 |     └─ widgets              #  背包 UI C++ 基类
48 |
49 | └─ Item/                     # === 物品系统 ===
50 |     └─ FPSItemEffectData    #  消耗品效果（DataAsset）
51 |
52 | └─ Level/                    # === 关卡 ===
53 |     └─ FPSLevelFlowTypes    #  Raid 流程枚举/结构
54 |     └─ FPSExtractionZone    #  撤离区域
55 |     └─ FPSPlayerStart       #  带阵营的出生点
56 |
57 | └─ UI/                       # === UI C++ 基类 ===
58 |     └─ FPSHUDWidget         #  HUD
59 |     └─ FPSCrosshairWidget   #  准星
60 |     └─ FPSMenuConfig        #  菜单配置
61 |     └─ FPSMenuTypes         #  菜单枚举/结构
62 |     └─ Menu/                #  菜单 widget 基类
63 |         └─ FPSMenuWidgetBase #  菜单基类（显示/隐藏动画）
64 |         └─ FPSMainMenuWidget #  主菜单
65 |         └─ FPSPauseMenuWidget #  暂停菜单
66 |         └─ FPSSettingsWidget #  设置
67 |         └─ FPSMapSelectWidget #  地图选择
68 |         └─ FPSLoadoutWidget  #  装备/结算
69 |         └─ FPSScoreboardWidget #  记分板（数据接口）
70 |
71 | └─ System/                   # === 全局子系统 ===
72 |     └─ FPSMenuSubsystem      #  菜单栈管理（GameInstanceSubsystem）

```

开发过程

整个项目从 UE5 First Person 模板起步，分阶段迭代。以下按时间顺序记录每个阶段的技术思路和踩坑经验。

阶段一：单机战斗原型

1.1 GAS 接入

项目最先要解决的是战斗属性管理。我没有用传统的 `TakeDamage` + 手写属性方式，而是直接上了 GAS (Gameplay Ability System)，原因是后期 Buff/Debuff、冷却、网络同步这些需求 GAS 天然支持，如果前期用土方法后面必然要重构。

第一步是在 `FPS.Build.cs` 里加上 `GameplayAbilities`、`GameplayTags`、`GameplayTasks` 三个模块依赖。然后按 GAS 标准三件套来搭：

- `UFPSAbilitySystemComponent`：继承 `UAbilitySystemComponent`，封装了 `GrantAbility`、`ApplyEffectToSelf` 等高频操作，避免业务层到处写模板代码。一开始 `ReplicationMode` 设的 `Full`，后面改多人的时候切成了 `Mixed`。
- `UFPSAttributeSetBase`：属性集基类，主要是抽出了 `ClampAttribute` 和 `AdjustAttributeForMaxChange` 两个工具方法，所有具体属性集继承它。
- `UFPSCombatAttributeSet`：实际战斗属性——Health/MaxHealth、Armor/MaxArmor、Stamina/MaxStamina、MovementSpeed，外加两个 Meta 属性 `IncomingDamage` 和 `IncomingHeal`。Meta 属性不复制，只在 `PostGameplayEffectExecute` 里做一次性消费。

伤害流程设计为：**GE 设置 `IncomingDamage` → `PostGameplayEffectExecute` 触发 →**

`HandleDamage` 计算护甲吸收 → 扣血 → `CheckDeath`。护甲吸收的公式是：先用 `Min`(当前护甲, 伤害) 算出护甲能挡多少，乘以 `ArmorReduction` (默认 0.5) 得到实际减免量，剩下的打到血上。这样护甲 100 + 减免 50% 的情况下，受到 100 伤害实际只扣 50 血和 100 甲。

踩过一个坑：`PreAttributeChange` 里做 `Clamp` 只影响「查询值」不影响「实际值」，真正要 `Clamp` 必须在 `PostGameplayEffectExecute` 里手动 `SetHealth`。官方文档对这个说得含糊，Lyra 源码里也是在 `Post` 里做的。

1.2 GameplayTags 注册

所有 Tag 走 Native 注册 (`FFPSGameplayTags` 单例)，在模块 `StartupModule` 时调

`InitializeNativeTags`。这样 C++ 侧引用 Tag 不需要写字符串，直接用

`FFPSGameplayTags::Get().State_Dead` 这种强引用。Tag 分了五大类：State、Ability、Effect、Event、Input。后面加 PVP 的时候又补了 `Team.A / Team.B / Team.None` 三个。

1.3 武器系统

武器做成了数据驱动架构：

- `UFPSWeaponDataAsset` (DataAsset)：配置所有武器参数——伤害、射速、射程、衰减、散布、弹夹、音效、特效、动画蒙太奇、GAS 能力类。策划或者 Lua 层只需要配 DataAsset，不用碰 C++。
- `AFPSWeaponBase` (Actor)：运行时武器实例。持有 DataAsset 引用，管理状态机 (`Idle/Firing/Reloading/Equipping/Unequipping`)、弹药、散布、射线检测、伤害应用。

散布系统：每次射击调 `IncreaseSpread` 增加散布角度，Tick 里 `UpdateSpread` 按 `SpreadRecoveryRate` 线性恢复到 `BaseSpread`。散布通过在瞄准方向上叠加随机锥形偏移实现，用球坐标转换保证均匀分布。

武器装备时通过 GAS 给角色 Grant 三个能力 (`Fire/Reload/Melee`)，卸下时 Clear，这样输入-能力的映射完全由 GAS 管理，武器切换时能力自动跟着换。

1.4 GAS 能力实现

四个能力都继承 `UFPSGameplayAbility`（自定义基类，加了 `ActivationPolicy` 枚举和 `Helper` 方法）：

- **GA_WeaponFire**：支持单发和自动两种模式。自动模式用 `TimerHandle` 做 `AutoFireTick`，`InputReleased` 时停止。每次 Fire 都通过 `Character` 拿到 `CurrentWeapon` 调它的 `TryFire`。
- **GA_WeaponReload**：激活时加 `State.ReloadTag`，这个 Tag 会 Block Fire 能力。完成/取消时移除。
- **GA_WeaponMelee**：球形扫描（`SphereTrace`），范围 150、半径 30、伤害 50。
- **GA_UseItem**：支持即时和延迟使用，可以 Apply GE 或 Grant Ability。

阶段二：UI 菜单系统 + UnLua 集成

2.1 菜单架构

菜单系统的核心是 `UFPSMenuSubsystem`（`UGameInstanceSubsystem`），好处是生命周期跟 `GameInstance` 走，不会因为关卡切换丢失状态。它维护了一个菜单栈，用 `PushMenu` / `PopMenu` / `PopAllMenus` 管理菜单层级，同时控制输入模式（Game Only / UI Only / Game And UI）。

C++ 侧为每个菜单创建了一个 Widget 基类（继承 `UFPSMenuWidgetBase`），提供显示/隐藏动画的骨架和 `NativeOnMenuShown` 等回调钩子。但实际的 UI 布局、按钮事件、动画表现全部在 Lua 做。

2.2 UnLua 绑定策略

UnLua 的绑定规则是：Widget Blueprint 的 Lua 脚本路径对应 `Content/Script/` 下的文件。比如 `WBP_MainMenu` 对应 `Content/Script/UI/Menu/WBP_MainMenu.lua`。C++ 基类暴露 `BlueprintCallable` 方法，Lua 侧直接调用，不需要额外绑定代码。

选 UnLua 而不是纯蓝图的原因：**热更新**。Lua 文件可以不重新打包就替换，对于 UI 和玩法逻辑频繁迭代的场景非常关键。C++ 只负责不会变的框架层，Lua 负责会变的表现层和逻辑层。

2.3 网格背包系统

背包单独做了一套：

- **UInventoryGridComponent**：纯逻辑组件，维护一个二维格子数组，处理物品放置、旋转、碰撞检测、堆叠。这部分全部 C++ 实现，因为涉及大量数组操作，Lua 性能不够。
- **UItemDataManager**：物品数据加载和查询，从 `DataTable` 或 `DataAsset` 读取。
- **Widget 层**：`UInventoryGridWidget` 和 `UItemIconWidget` 提供 C++ 基类，Lua 侧 `WBP_InventoryGrid.lua` 实现拖拽、右键菜单、Tooltip 等交互。

拖拽的实现踩了个坑：UMG 自带的 Drag & Drop 系统在 Lua 侧用起来很别扭，最后改成了自己在 Lua 里 Track 鼠标位置 + 手动移动 Widget 的方案，体验更好。

阶段三：Raid 单人流程（PVE 原型）

在做 PVP 之前，先做了一个 Raid（塔科夫式）的单人流程验证：

- **AFPSGameMode**（当时继承 `AGameModeBase`）：管理 Raid 计时、状态机（`WaitingToStart` → `InProgress` → `Extracting` → `Extracted/Failed/TimedOut`）。
- **AFPSExtractionZone**：撤离区域，Box Trigger + 进度条，玩家在区域内站够时间就撤离成功。
- **FFPSRaidResult**：结算数据结构，记录成功/失败、时间、伤害、击杀、经验。

这套流程主要是验证 GameMode → MenuSubsystem → UI 的整条链路能跑通。验证完之后才开始做 PVP。

阶段四：PVP 联网改造

这是改动量最大的一个阶段。核心挑战是把一个单机项目改成支持多人对抗，需要动的地方非常多。

4.1 阵营系统 (Team/)

新建了 `Team/` 目录，从零开始：

`FPSTeamTypes.h`：定义 `EFPSTeam` (None/TeamA/TeamB) 和 `EFPSMatchState` (WaitingForPlayers/Countdown/InProgress/RoundEnd/GameOver) 两个枚举，以及 `UFPSTeamStatics` 工具函数库。`AreEnemies` / `AreFriendly` 这些判断函数看起来简单，但统一收到一个地方后业务代码干净很多。

`AFPSPlayerState`：继承 `APlayerState`，这是每个玩家的「身份证」。放了 Team、Kills、Deaths、Assists、Score、DamageDealt、DamageTaken 七个复制属性。所有写操作都加了 `HasAuthority` 检查，只有服务端能改。Team、Kills、Score 三个有 `OnRep` 回调和对应的委托，方便客户端 UI 监听。

设计决策：**PlayerState 做属性的 Owner 而不是 Character**。原因是多人游戏中角色会反复销毁重建（死亡/重生），但 PlayerState 始终存在。如果 Stats 跟着 Character 走，每次重生都要迁移数据。

`AFPSGameState`：继承 `AGameStateBase`，存全局数据——MatchState、TeamAScore/TeamBScore、MatchTimeRemaining、ScoreToWin（默认 30）、MatchTimeLimitSeconds（默认 600）。全部 Replicated，客户端随时能查。

4.2 GameMode 重写

这是改动最激烈的文件。基类从 `AGameModeBase` 换成了 `AGameMode`。两者的区别：`AGameMode` 有内置的 Match State 状态机 (WaitingToStart → InProgress → WaitingPostMatch → ...) 和 `ReadyToStartMatch` 等虚函数，比较适合 PVP 这种有明确开始/结束的玩法。

核心逻辑：

- **阵营分配**：`PostLogin` 里调 `ChooseTeamForPlayer`，策略是自动平衡——遍历 `GameState->PlayerArray` 统计两队人数，新玩家分到人数少的队。
- **出生点选择**：重写 `ChoosePlayerStartImplementation`，先收集所有 `AFPSPlayerStart`，过滤出和玩家同阵营的，随机选一个。没有阵营出生点就 fallback 到普通 `APlayerStart`。
- **击杀处理**：`HandlePlayerDeath(Victim, Killer)` 一站式处理——Victim 加死亡，Killer 加击杀和分数，GameState 加队伍分数，检查胜利条件，启动重生定时器。
- **重生**：`RespawnPlayer` 在延迟到期后调用，内部调 `FindPlayerStart` → `RestartPlayerAtPlayerStart`。比赛结束后不再重生。
- **胜利条件**：分数达标或时间耗尽。时间耗尽时按分数判，平局就 Winner = None。
- **比赛流程**：`ReadyToStartMatch` 要求至少 MinPlayersToStart 人 → `HandleMatchHasStarted` 触发 Countdown → 倒计时结束进入 InProgress → 计时器跑到 0 或分数达标触发 GameOver。

踩坑记录：`AGameMode` 的 `ReadyToStartMatch` 默认返回 true，会导致单人进去就直接开始。必须重写为检查人数。另外 `RestartPlayerAtPlayerStart` 会给角色重置控制器，如果 Character 的 `PossessedBy` 里没有重新 `InitAbilityActorInfo`，GAS 就断了。

4.3 Character 网络化改造

改动点：

1. **IGenericTeamAgentInterface**：让 Character 多继承这个接口，实现 `GetGenericTeamId`（从 `PlayerState` 取 `Team`）和 `GetTeamAttitudeTowards`（同队 `Friendly`，异队 `Hostile`，无队 `Neutral`）。之所以用引擎的 `Team` 接口而不是自己判断，是因为 AI 系统（`AIModule`）也认这个接口，后面加 AI Bot 的时候可以无缝兼容。`FPS.Build.cs` 因此加了 `AIModule` 依赖。
2. **第三人称网格 Mesh3P**：第一人称的 `Mesh1P` 设了 `OnlyOwnerSee`，自己能看到自己的手臂但别人看不到。新加的 `Mesh3P` 设了 `OwnerNoSee`，自己看不到但别人能看到完整人物模型。这是 FPS 多人游戏的标准做法。
3. **ASC Owner 改为 PlayerState**：单机时 `InitAbilityActorInfo(this, this)` 就够了。多人时必须改成 `InitAbilityActorInfo(PlayerState, this)`，因为 GAS 的复制机制依赖 Owner 的网络通道（`NetConnection`）。如果 Owner 是 Character，那角色销毁重建后 ASC 的复制状态就丢了。改成 `PlayerState` 后就没这个问题。
4. **复制属性**：`GetLifetimeReplicatedProps` 里注册了 `CurrentWeapon` 和 `bDead`。
`CurrentWeapon` 复制是为了让其他客户端知道这个人拿的什么枪（后续做第三人称武器显示用）。
5. **初始化时机**：服务端在 `PossessedBy` 里初始化 ASC，客户端在 `OnRep_PlayerState` 里初始化。
能力和效果只在服务端 Grant/Apply（`HasAuthority` 检查）。
6. **死亡通知 GameMode**：`OnDeath` 回调里，服务端从 Killer（`AActor*`）Cast 到 `AFPSCharacter` 再取 `PlayerState`，然后调 `GameMode->HandlePlayerDeath`。

4.4 武器网络同步

一开始方案是单独建 `FPSWeaponComponent` 放 RPC，后来发现没必要多一层间接，直接在 `AFPSWeaponBase` 里加就行：

- **ServerFire**（`Server, Reliable, WithValidation`）：客户端调用，传 `MuzzleLocation` 和 `FireDirection`。`_Validate` 函数检查客户端报的枪口位置和服务端计算的差距不超过 200 单位（防作弊）。`_Implementation` 在服务端执行射线检测和伤害。
- **MulticastFireEffects**（`NetMulticast, Unreliable`）：服务端广播给所有客户端播放枪口火焰、弹道、命中特效。本地玩家已经预播了所以跳过（`IsLocallyControlled` 判断）。用 `Unreliable` 是因为特效丢几帧也无所谓，不值得占可靠通道。

`Fire()` 的流程改为：本地客户端先播效果（手感不延迟），然后发 `ServerFire` → 服务端做权威判定 → `Multicast` 给其他人播效果。

友伤过滤：在 `ApplyDamage` 最前面加了 `IsFriendlyTarget` 检查，内部调 `OwnerCharacter->GetTeamAttitudeTowards(*TargetChar)`，如果是 `Friendly` 直接 return。

构造函数里加了 `bReplicates = true`，武器 Equip 时调 `SetOwner(Character)` 保证 RPC 的网络通道正确。

4.5 伤害统计回流 PlayerState

在 `FPSCombatAttributeSet::HandleDamage` 里，算完最终伤害后，把数值回写到双方 `PlayerState`：

- ```
1 Victim->PlayerState->AddDamageTaken(RemainingDamage)
2 Instigator->PlayerState->AddDamageDealt(RemainingDamage)
```



从 ASC 拿到 AvatarActor (Character) , 再从 Character 拿 PlayerState。Instigator 从 EffectSpec.GetContext().GetEffectCauser() 取, 这个值在武器 ApplyDamage 时通过 ContextHandle.AddSourceObject(this) 设进去的。击杀统计不在此处做——CheckDeath 触发 OnDeath 委托, Character 收到后通知 GameMode, GameMode 统一处理 AddKill。这样单一职责更清晰。

## 4.6 阵营出生点

AFPSPlayerStart 非常简单——继承 APlayerStart, 加一个 EFPSTeam Team 属性, EditAnywhere 让关卡设计师在编辑器里直接设。GameMode 的 ChoosePlayerStart\_Implementation 会按阵营筛选。

## 4.7 PlayerController 会话管理

给 Lua 层暴露了三个 BlueprintCallable:

- HostGame(MapName, MaxPlayers): 拼接 ?listen?MaxPlayers=N 参数, 调 ServerTravel。Listen Server 模式下 Host 既是服务器又是客户端。
- JoinGame(Address): 调 ClientTravel(Address, TRAVEL\_Absolute), 直连 IP:Port。LAN 场景下不需要 OnlineSubsystem, UE 内置的 Session 就够。
- LeaveGame(): 服务端调 ReturnToMainMenuHost, 客户端调 ClientTravel 回主菜单地图。

另外加了两个 Client RPC:

- ClientShowKillFeed: 服务端推送击杀消息到客户端, Lua 监听后渲染 Kill Feed UI。
- ClientOnMatchStateChanged: 比赛状态变化通知, Lua 监听后做倒计时、胜负画面等。

为什么不用 GameState 的 OnRep 而单独发 RPC? 因为 OnRep 只告诉你「值变了」, 不带额外参数 (比如 Winner 是谁)。RPC 可以附带上下文信息。

## 4.8 记分板数据接口

UFPSScoreboardWidget 作为 C++ 基类, 提供纯数据查询:

- GetAllPlayerStats(): 遍历 GameState->PlayerArray, 每个 PlayerState 提取 Name/Team/K-D-A/Score/Ping, 按分数降序排列返回 TArray<FFPSPlayerScoreInfo>。
- GetTeamScore(Team)、GetMatchTimeRemaining()、GetMatchState(): 直接从 GameState 取。

Lua 侧拿到数组后自由渲染, C++ 不管布局。

## 阶段五: Lua UI 层实现

C++ 框架铺完后, 所有 UI 和交互逻辑在 Lua 里实现:

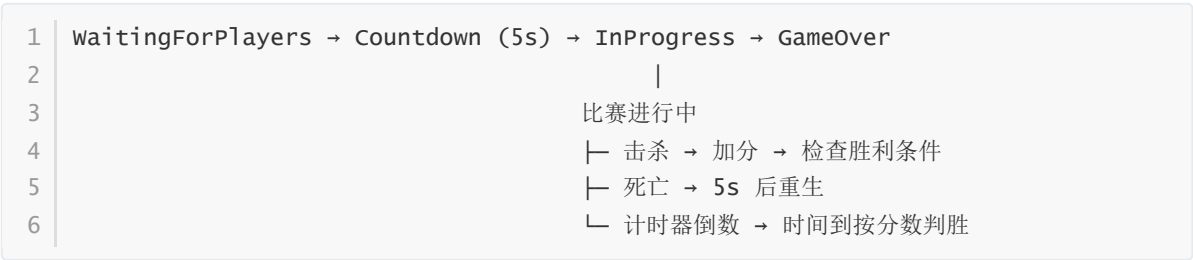
| 脚本                | 做了什么                                             |
|-------------------|--------------------------------------------------|
| WBP_MainMenu.lua  | 主菜单布局、Host/Join/Settings/Exit 按钮事件、IP 输入框        |
| WBP_PauseMenu.lua | 暂停菜单、继续/设置/退出按钮                                  |
| WBP_Settings.lua  | 画质/音量/灵敏度滑块, 调用 FPSMenuSubsystem 的 Get/Set/Apply |

| 脚本                    | 做了什么                                     |
|-----------------------|------------------------------------------|
| WBP_MapSelect.lua     | 地图列表渲染、选中态、确认开始                          |
| WBP_Loadout.lua       | 装备选择 + Raid 结算复用，通过 EFPSLoadoutMode 区分   |
| WBP_HUD.lua           | 血条/护甲/弹药/准星/比分/倒计时/击杀通知，监听 GAS 属性变化和 RPC |
| WBP_InventoryGrid.lua | 网格背包拖拽、右键菜单、物品 Tooltip                   |

Lua 层的优势在实际开发中很明显：改 UI 不用重新编译 C++，编辑器里改完 Lua 文件 `Ctrl+S` 就能看到效果。

## 核心系统

### PVP 比赛流程



- **阵营分配**：自动平衡，新玩家加入人少的队
- **胜利条件**：先到 30 分，或 10 分钟后分高者胜
- **重生**：死亡后 5 秒在己方出生点随机重生

### 网络架构

| 概念        | 实现                                                         |
|-----------|------------------------------------------------------------|
| 模型        | Listen Server（一台做 Host，其余 Join）                            |
| 连接        | LAN 直连 IP:Port                                             |
| 射击        | 客户端 → ServerFire RPC → 服务端射线检测 + 伤害 → MulticastFireEffects |
| 状态同步      | PlayerState / GameState 属性复制                               |
| GAS 复制    | Mixed 模式（GE 只复制给 Owner，Cue 广播所有人）                          |
| ASC Owner | PlayerState（保证 Controller 切换时 ASC 不丢失）                     |



# 战斗系统

- **武器**：数据驱动 (DataAsset) ， 支持步枪/手枪/霰弹枪/冲锋枪/狙击枪/近战
- **伤害计算**：基础伤害 × 距离衰减 × 爆头倍率 → 护甲吸收 → 扣血
- **友伤过滤**：通过 `IGenericTeamAgentInterface` 判断阵营关系，友方不受伤
- **弹道**：射线检测 + 散布系统（射击增加散布，停火恢复）
- **属性**：GAS 驱动 (Health / Armor / Stamina) ， 全部网络复制

# Lua 热更层

通过 UnLua 插件将 Widget Blueprint 绑定到 Lua 脚本，实现：

| Lua 脚本                             | 功能                        |
|------------------------------------|---------------------------|
| <code>WBP_MainMenu.lua</code>      | 主菜单 (Host/Join/退出)        |
| <code>WBP_PauseMenu.lua</code>     | 暂停菜单                      |
| <code>WBP_Settings.lua</code>      | 设置 (画质/音量/灵敏度)            |
| <code>WBP_MapSelect.lua</code>     | 地图选择                      |
| <code>WBP_Loadout.lua</code>       | 装备选择 / 战局结算               |
| <code>WBP_HUD.lua</code>           | 战斗 HUD (血量/弹药/计时/比分/击杀通知) |
| <code>WBP_InventoryGrid.lua</code> | 网格背包 (拖拽/旋转)              |

C++ 只提供数据接口，不做任何 UI 布局和表现逻辑。

# 快速开始

## 环境要求

- Unreal Engine 5.4
- Visual Studio 2022 / Rider
- Windows 10/11

## 编译运行

```
1 # 1. 用 UE5.4 打开 FPS.uproject
2 # 2. 等待 Shader 编译完成
3 # 3. 编译 C++ (Ctrl+Alt+F11 或 Build 按钮)
```

## PIE 双人测试

1. 编辑器 → Play → **Number of Players = 2**
2. Net Mode 选 **Play As Listen Server**
3. 两个窗口应分到不同阵营，互相可造成伤害

## LAN 测试

```
1 # 主机
2 在主菜单点击 Host，或调用 PlayerController->HostGame("MapName", 8)
3
4 # 客户端
5 在主菜单输入主机 IP，或调用 PlayerController->JoinGame("192.168.x.x:7777")
```

## 关卡配置

1. **World Settings** → GameMode Override → 选择 **AFPSGameMode** (或其蓝图子类)
2. 在地图中放置 **FPSPPlayerStart** :
  - 红队出生点: Team = TeamA, 放 3-5 个
  - 蓝队出生点: Team = TeamB, 放 3-5 个
3. 确保 **BP\_FirstPersonCharacter** 的 **Mesh3P** 设置了第三人称骨骼网格

## 关键 C++ 接口 (供 Lua 调用)

### 会话管理 (FPSPPlayerController)

```
1 -- 开房
2 self:HostGame("/Game/Maps/Arena", 8)
3 -- 加入
4 self:JoinGame("192.168.1.100:7777")
5 -- 离开
6 self:LeaveGame()
```

### 记分板数据 (FPSScoreboardWidget)

```
1 local stats = self:GetAllPlayerStats() -- TArray<FFPSPPlayerScoreInfo>
2 local teamAScore = self:GetTeamScore(EFPSTeam.TeamA)
3 local timeStr = self:GetFormattedTimeRemaining() -- "05:30"
4 local state = self:GetMatchState() -- EFPSSMatchState
```

### 阵营工具 (UFPSTeamStatics)

```
1 local isEnemy = UFPSTeamStatics.AreEnemies(teamA, teamB)
2 local color = UFPSTeamStatics.GetTeamColor(team)
3 local name = UFPSTeamStatics.GetTeamDisplayName(team)
```

## 比赛事件 (Client RPC → Lua 监听)

```
1 -- FPSPlayerController
2 function M:ClientShowKillFeed(KillerName, VictimName)
3 -- 显示击杀通知
4 end
5
6 function M:ClientOnMatchStateChanged(NewState, Winner)
7 -- 比赛状态变化 (倒计时/开始/结束)
8 end
```

## 配置参数

| 参数                    | 位置           | 默认值 | 说明         |
|-----------------------|--------------|-----|------------|
| ScoreToWin            | FPSGameState | 30  | 获胜所需分数     |
| MatchTimeLimitSeconds | FPSGameState | 600 | 比赛时间限制 (秒) |
| RespawnDelay          | FPSGameMode  | 5.0 | 重生延迟 (秒)   |
| KillScore             | FPSGameMode  | 1   | 每次击杀得分     |
| CountdownDuration     | FPSGameMode  | 5.0 | 开局倒计时 (秒)  |
| MinPlayersToStart     | FPSGameMode  | 2   | 最少开局人数     |

## 依赖模块

```
1 Core, CoreUObject, Engine, InputCore, EnhancedInput,
2 UMG, Slate, SlateCore,
3 GameplayAbilities, GameplayTags, GameplayTasks,
4 Niagara, AIModule
```

插件: `GameplayAbilities`, `GamePlay`, `UnLua`, `ModelingToolsEditorMode` (Editor only)

## License

Private project. All rights reserved.